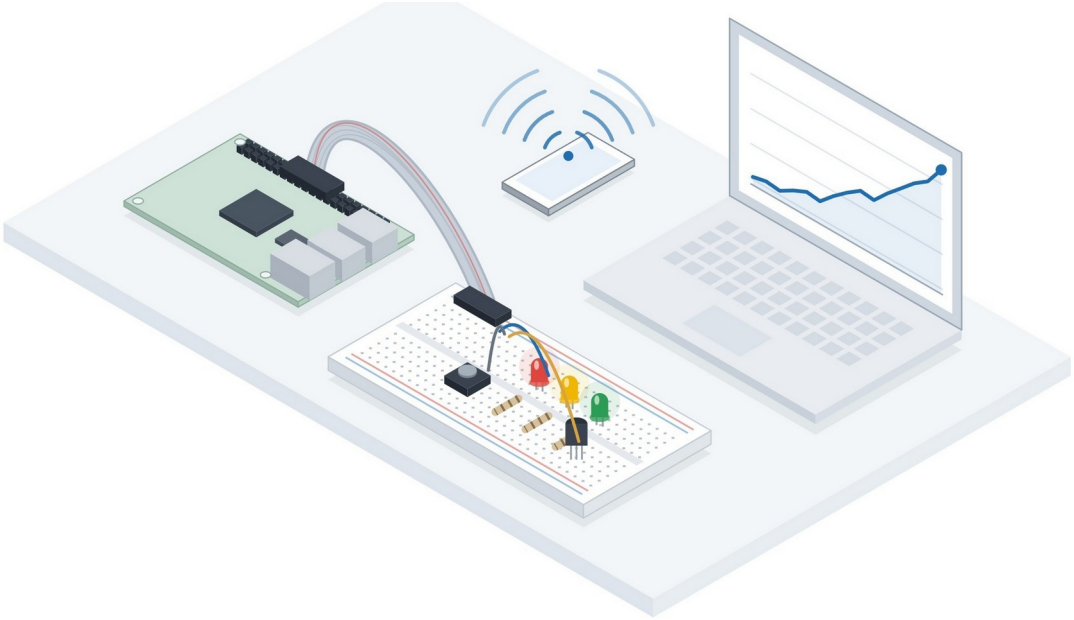
Trabajo final de Informática y Programación

Informática y Programación 2026

Estación de monitoreo de temperatura con Raspberry Pi



SOBOLEOSKY Ian · Legajo 14794
Ingeniería en Mecatrónica
Modalidad Raspberry Pi · Trabajo individual

Índice

Introducción general....3
Parte A....5
Introducción....5
Requerimientos y restricciones....6
Circuito utilizado....7
Algoritmo general de alto nivel....9
Diagramas de flujo de la solución....11
Código en Python....14
Registro de pruebas previas a la presentación....17
Parte B....19
Introducción....19
Requerimientos y restricciones....19
Estructura de datos y memoria dinámica....19
Algoritmo general de alto nivel....20
Diagrama de flujo....21
Librerías utilizadas....21
Descripción de las subrutinas....22
Captura de pantalla demostrando el funcionamiento....22
Parte C....24
Introducción....24
Requerimientos y restricciones....24
Librerías adicionales utilizadas....24
Algoritmo general de alto nivel....26
Captura de pantalla demostrando el funcionamiento....26
Adicionales....27
Python de 32 bits en Windows....27
Acceso remoto por SSH y mDNS....27
Prueba final: notebook del coloquio y hotspot....28
Conclusiones....29
Cumplimiento de especificaciones....29
Consideraciones finales sobre el funcionamiento....29

Introducción general

En este trabajo armé una estación para medir temperatura con una Raspberry Pi 3 Model B, un sensor DS18B20, tres LEDs y un pulsador. La idea es tomar lecturas, guardarlas y después poder analizarlas. El trabajo está dividido en las tres partes de la consigna:

1. Parte A (Python). La Raspberry lee la temperatura cada 3,5 s, muestra el promedio de las últimas 5 lecturas y señala la tendencia con los LEDs. El pulsador permite cambiar el tiempo entre lecturas o terminar el programa. Los datos se guardan en registro_temperaturas.csv. El ciclo elegido queda en memoria; al reiniciar vuelve a 3,5 s.
2. Parte B (C). El programa lee el CSV y calcula la cantidad de lecturas, la mínima y la máxima con fecha y hora, la media, la mediana, la moda y el desvío estándar.
3. Parte C (Python). La Raspberry envía las temperaturas a una notebook por TCP. La notebook trabaja como servidor en el puerto 30103 y muestra un gráfico con matplotlib.

La Raspberry se maneja desde la notebook por SSH, sin conectarle un monitor. La Parte B usa el archivo que genera la Parte A. Para el gráfico se usa la conexión de red, pero la estación puede seguir midiendo aunque la PC no esté conectada.

Archivo	Parte	Dónde corre	Líneas	Qué hace
sensor.py	A	Raspberry	38	Módulo con leer_temperatura()
estacion.py	A y C	Raspberry	269	Estación completa + cliente TCP
estadistica.c	B	Raspberry	176	Análisis estadístico del CSV
grafico.py	C	PC	75	Servidor TCP y gráfico en tiempo real

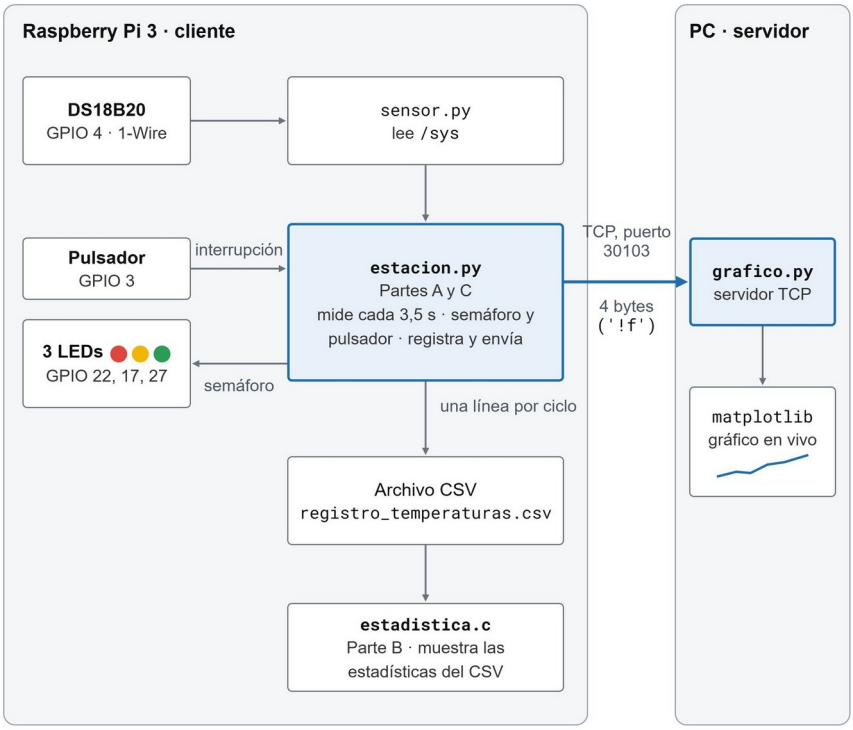


Figura 1. *¿Dónde corre cada proceso? Programas, archivo y conexión.*

Parte A

Se plantea realizar una estación de monitoreo y seguimiento de temperatura, la cual debe cumplir con los siguientes requisitos:

- Debe realizar la lectura de temperatura en ciclos de X segundos. Por defecto el ciclo es de 3.5 segundos. Al iniciar, el equipo debe comenzar a registrar datos con esta frecuencia.
- En cada ciclo la estación muestra por consola la temperatura que representa la última temperatura leída y el promedio de los últimos N de los eventos (por defecto N = 5).
- El ciclo se configura mediante un pulsador: al presionarlo, comienza el conteo en milisegundos y, al soltarlo, se finaliza. Se notificará al usuario el cambio en la configuración; el equipo comenzará a registrar datos inmediatamente a la nueva frecuencia. El ciclo mínimo es de 2.5 segundos (pulsando desde 1" hasta 2.5") y el ciclo máximo es de 10 (pulsando desde 10" en adelante). Una vez concluido el cálculo del ciclo se informa por pantalla el nuevo tiempo del ciclo y se registra a partir de este momento mediante la nueva frecuencia.
- El mismo pulsador se utiliza para finalizar el programa de monitoreo, esto es, si el pulsador se mantiene apretado por menos de 1 segundo.
- La estación además posee un semáforo con 3 leds, los cuales indican la tendencia de cambio de la temperatura:
  - Un led rojo se enciende cuando el valor de la lectura en un ciclo supera en un X% el promedio de los últimos N eventos (tendencia en alta).
  - Un led amarillo indica que la temperatura se mantiene estable, es decir, se encuentra dentro del promedio (±X%).
  - Un led verde enciende cuando la temperatura disminuye en un X% del promedio de los últimos N eventos (tendencia en baja).
  - Cada vez que un led enciende, los otros dos se apagan.
  - Los 3 leds de tendencia se mantendrán prendidos cuando **no** se tenga la suficiente información para analizar la tendencia, hasta que se dispongan de al menos los N eventos necesarios para calcular el promedio.
  - Mientras el pulsador esté presionado, una vez por segundo se deberán destellar durante 50 ms todos los leds indicando los segundos; luego se prenderá el led de la tendencia actual, o todos si no la hubiere.
  - Mientras la estación esté midiendo se deberán destellar durante 50 ms todos los leds indicando el fin del ciclo y la captura realizada; luego se prenderá el led de la tendencia actual, o todos si no la hubiere.

- Una última tarea de la estación es ir registrando en un archivo, por cada ciclo: la lectura de temperatura, la tendencia detectada (para cada uno de los 4 estados posibles) y la fecha y hora de la lectura.

Sobre la resolución: se debe realizar un análisis y diseño algorítmico previo (entradas, salidas, información relevante, requerimientos y restricciones) y un algoritmo general de alto nivel. El ejercicio debe ser codificado en Python.

Introducción

Esta parte se encarga de medir, manejar los LEDs y el pulsador, y guardar las lecturas. Separé el código en sensor.py, que lee el DS18B20, y estacion.py, que controla el resto. Para iniciarlo en la Raspberry uso cd ~/tpi/simple y después python3 estacion.py.

Requerimientos y restricciones

Primero ordené las entradas, las salidas y las variables que iba a necesitar. En las tablas se indica también el tipo de dato.

Tipo	Dato	Tipo de dato	Origen / destino
Entrada	Temperatura	str → int → float (°C)	Texto t=31312 del archivo w1_slave (DS18B20, GPIO 4); se convierte a entero y se divide por 1000.
Entrada	Nivel del pulsador	int (0 o 1)	GPIO 3 con pull-up: suelto = 1, apretado = 0.
Entrada	Tiempo apretado	float (s)	Resta de dos time.time(): al soltar menos al apretar.
Entrada	IP de la PC (Parte C)	str o None	Argumento al correr: sys.argv[1].
Salida	Línea por consola	str	Hora, temperatura, promedio de las últimas 5 y tendencia; avisos del pulsador.
Salida	Semáforo	int (0 o 1) por pin	LED rojo GPIO 22, amarillo GPIO 17, verde GPIO 27.
Salida	Registro	str (una línea de texto)	registro_temperaturas.csv: fecha,hora,temperatura,tendencia, con la temperatura en 3 decimales.
Salida	Temperatura a la PC	bytes (4, real de 32 bits)	Socket TCP, struct.pack('!f', T).

Datos que el programa guarda en memoria:

Variable	Tipo	Valor inicial	Para qué
ciclo	float	3.5	Segundos entre lecturas; lo cambia el pulsador.
ultimas	list de float	[]	Las últimas 5 temperaturas, para el promedio y la tendencia.
tendencia	str	'SIN_DATOS'	Estado del semáforo: ALTA, ESTABLE, BAJA o SIN_DATOS.
pulsando	bool	False	Si el pulsador está apretado.
instante_apretado	float	0.0	Cuándo se apretó, para medir el tiempo.
estado_pulsador	int	1	Último nivel estable, para el antirrebote.
en_marcha	bool	True	False = terminar el programa.
conexion	socket o None	None	La conexión con la PC (Parte C).

Al principio de estacion.py están N = 5, X = 1.0 (%), los pines, el nombre del archivo y el puerto. Así quedan juntos los valores que se pueden cambiar.

Restricciones:

- El ciclo va de 2,5 s a 10 s, y la tendencia necesita N lecturas previas.
- El pulsador tiene que responder incluso mientras se lee el sensor o se espera la siguiente medición. Para resolverlo usé interrupciones e hilos.
- Cada lectura del sensor tarda unos 0,8 s, que están dentro del ciclo: hay que esperar solo lo que falta.
- Un pulsador mecánico rebota (vi cambios separados por unos 20 ms): hace falta antirrebote.
- Al terminar hay que cerrar el archivo y las conexiones, quitar la interrupción y liberar los GPIO.

Punto abierto	Qué elegí
Valor de X %	**X = 1 %**
Contra qué promedio se compara	Las **5 lecturas anteriores**, sin la actual
Promedio por consola	Últimas 5 **incluyendo** la actual
Los 4 estados	ALTA, ESTABLE, BAJA, SIN_DATOS

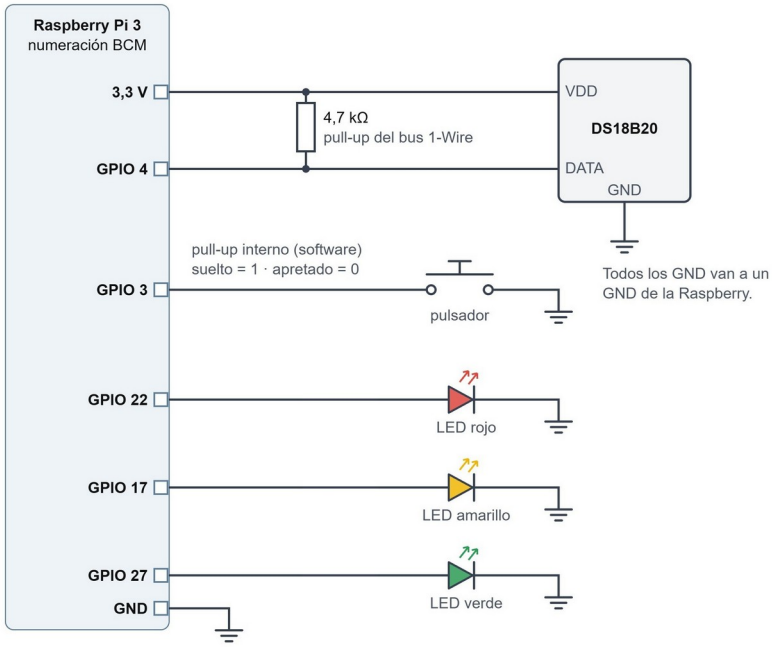
Punto abierto	Qué elegí
Destello con los 3 LEDs prendidos	Los **apaga** 50 ms
Límites del pulsador	< 1 s termina; 1–2,5 s → 2,5 s; 2,5–10 s → ese tiempo; ≥ 10 s → 10 s
Mientras se aprieta	**Se sigue midiendo**
Soltar durante una lectura	La lectura termina y **se mide otra vez enseguida**
Qué es un ciclo de 3,5 s	Del comienzo de una lectura al de la siguiente

Circuito utilizado

Usé el kit de la cátedra: Raspberry Pi 3 Model B, protoboard y Pi Cobbler+ (Adafruit), un DS18B20, tres LEDs (conectados sin resistencia en serie), un pulsador, una resistencia de 4,7 kΩ (pull-up del bus 1-Wire) y una de 10 kΩ. Antes de escribir la estación confirmé cada conexión con un programa de prueba (`prueba_hw.py`).

Componente	GPIO (BCM)	Lógica
LED rojo / amarillo / verde	22 / 17 / 27	Salida; 1 = prendido
Pulsador	3	Entrada con pull-up: suelto = 1, apretado = 0
DS18B20 (datos)	4	Bus 1-Wire, lo maneja el kernel

Usé la numeración BCM (`GPIO.setmode(GPIO.BCM)`), que es la del chip y la que figura en el Cobbler. Con el pull-up activado (`pull_up_down=GPIO.PUD_UP`) el pin lee 1 con el pulsador suelto; al apretarlo lo une a masa y lee 0.



Componente	GPIO (BCM)	Lógica
LED rojo	22	1 = prendido
LED amarillo	17	1 = prendido
LED verde	27	1 = prendido
Pulsador	3	pull-up: suelto = 1, apretado = 0
DS18B20 (DATA)	4	bus 1-Wire, pull-up de 4,7 kΩ

Figura 2. *Circuito esquemático (numeración BCM). Todos los GND van a un GND de la Raspberry.*

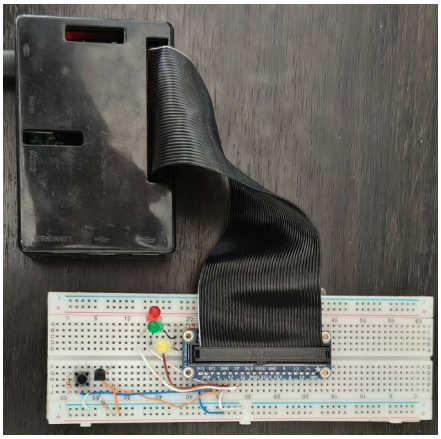


Figura 3. *La Raspberry en su case, unida al protoboard por el cable plano.*

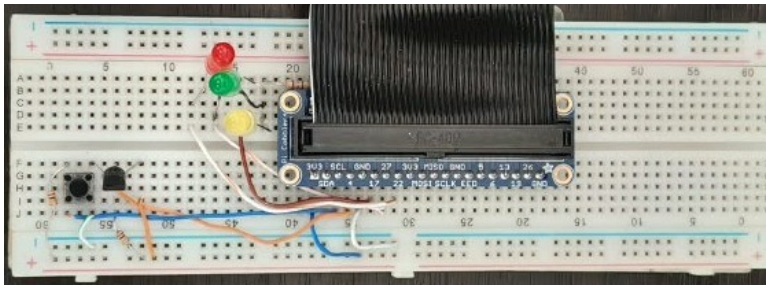


Figura 4. *El protoboard: arriba los LEDs rojo, verde y amarillo; abajo a la izquierda el pulsador y el DS18B20; a la derecha el Pi Cobbler+.*

Sensor utilizado: DS18B20

El DS18B20 es un sensor digital que se comunica por 1-Wire, con un solo cable de datos. Su rango es de −55 °C a 125 °C y tiene una precisión de ±0,5 °C entre −10 °C y 85 °C. En este trabajo se usó a 12 bits, con pasos de 0,0625 °C. Las lecturas tardaron cerca de 800 ms.

Cómo lo lee la Raspberry. No usé ninguna librería del sensor. El overlay `dtoverlay=w1-gpio,gpiopin=4` (en `/boot/firmware/config.txt`) hace que el kernel maneje el bus y muestre cada sensor como una carpeta en `/sys/bus/w1/devices/`; la mi es `28-0516b533b8ff` (`28` es la familia DS18B20). Adentro, `w1_slave` tiene dos líneas:

```
f5 01 4b 46 7f ff 0c 10 84 : crc=84 YES
f5 01 4b 46 7f ff 0c 10 84 t=31312
```

La primera línea termina en YES cuando el control de errores (CRC) es correcto. En la segunda aparece la temperatura en milésimas de grado: 31312 corresponde a 31,312 °C. El archivo w1_slave lo genera el sistema al consultar el sensor. Seguí el tutorial de la cátedra: buscar la carpeta con glob, comprobar YES y dividir el valor por 1000. En esta configuración los controladores se cargan con el overlay. Si la lectura falla, la función devuelve None y la estación saltea ese ciclo.

Algoritmo general de alto nivel

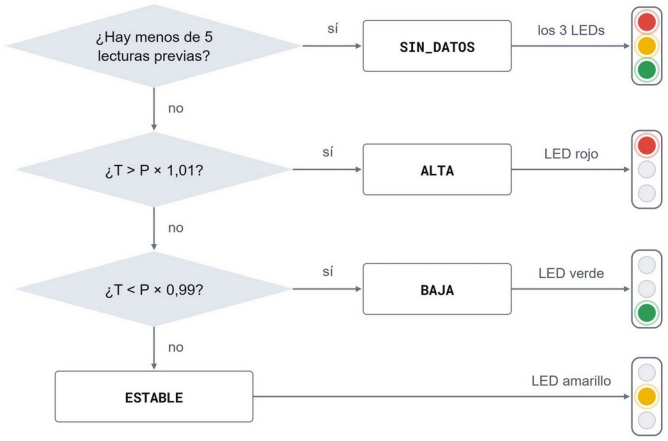
Programa principal

1. Configurar los pines (LEDs como salida, pulsador con pull-up) y prender los 3 LEDs. Registrar la interrupción del pulsador.
2. Abrir el CSV en modo agregar (si es nuevo, escribir el encabezado). Si se indicó la IP de la PC, lanzar el hilo de red.
3. En cada ciclo se guarda la hora de inicio y se limpia el evento despertar. Después se lee el sensor, se calcula la tendencia con las 5 lecturas anteriores y se actualiza el promedio. La lectura se muestra, se guarda y, si hay conexión, se envía a la PC. Por último se hace el destello y se espera el tiempo que falta. Si falla la lectura, se saltea el procesamiento y se respeta la espera.
4. Al salir (pulsador, Ctrl+C o error): cerrar todo en orden.

Cómo se accede al archivo. En Python, `open(nombre, modo)` define qué se puede hacer con el archivo:

Modo	Qué hace	Dónde lo usé
'r'	Solo lectura. Falla si el archivo no existe.	sensor.py lee w1_slave; en C, estadistica.c abre el CSV con fopen(ARCHIVO, "r").
'w'	Escritura. Crea el archivo o **borra** lo que tenía.	No lo usé: perderíamos las lecturas anteriores en cada ejecución.
'a'	Agregar. Crea el archivo si no existe y escribe siempre al final, sin borrar.	estacion.py abre el CSV con open(ARCHIVO, 'a'): los datos se acumulan entre ejecuciones.

Antes de abrir el CSV se comprueba con os.path.exists() si ya existe, para escribir el encabezado una sola vez. Después de cada registro se usa flush() para vaciar el búfer de Python; esto no garantiza por sí solo que el dato quede guardado ante un corte de luz. Los permisos que muestra ls -l son -rw-rw-r--: el dueño y el grupo pueden leer y escribir, y los demás solo leer.



El semáforo muestra el **cambio**, no el nivel: con la temperatura alta pero quieta vuelve a ESTABLE.

Figura 5. *El semáforo compara cada lectura con el promedio P de las 5 anteriores (X = 1 %).*

La interrupción se ejecuta cuando cambia el estado del pulsador. Primero comprueba que el programa no esté cerrando y espera a que el nivel se mantenga estable durante 50 ms. Al apretar guarda el instante e inicia los destellos. Al soltar calcula cuánto tiempo pasó: si fue menos de 1 s, termina; en los demás casos cambia el ciclo. También activa el evento despertar para interrumpir la espera del programa principal.

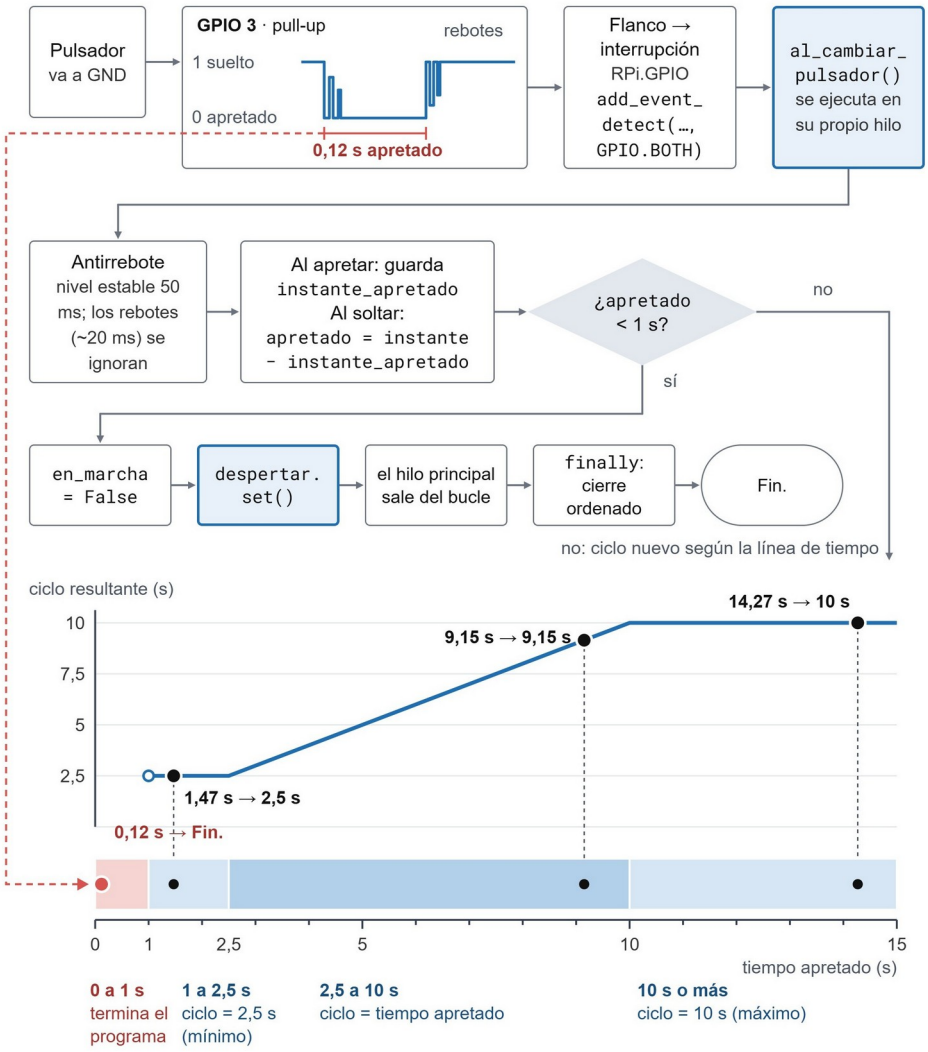


Figura 6. *Cómo la Raspberry recibe la señal del pulsador y decide: un toque termina, mantenerlo apretado cambia el ciclo. Los puntos negros son pruebas reales.*

Destellos por segundo: cada vez que se cumple un segundo más desde que se apretó, destellar 50 ms; terminar al soltar. **Red:** mientras no haya conexión, intentar conectarse a la PC (máximo 5 s); esperar 2 s y repetir.

Diagramas de flujo de la solución

Los diagramas muestran por separado el programa principal y las tareas de los otros hilos.

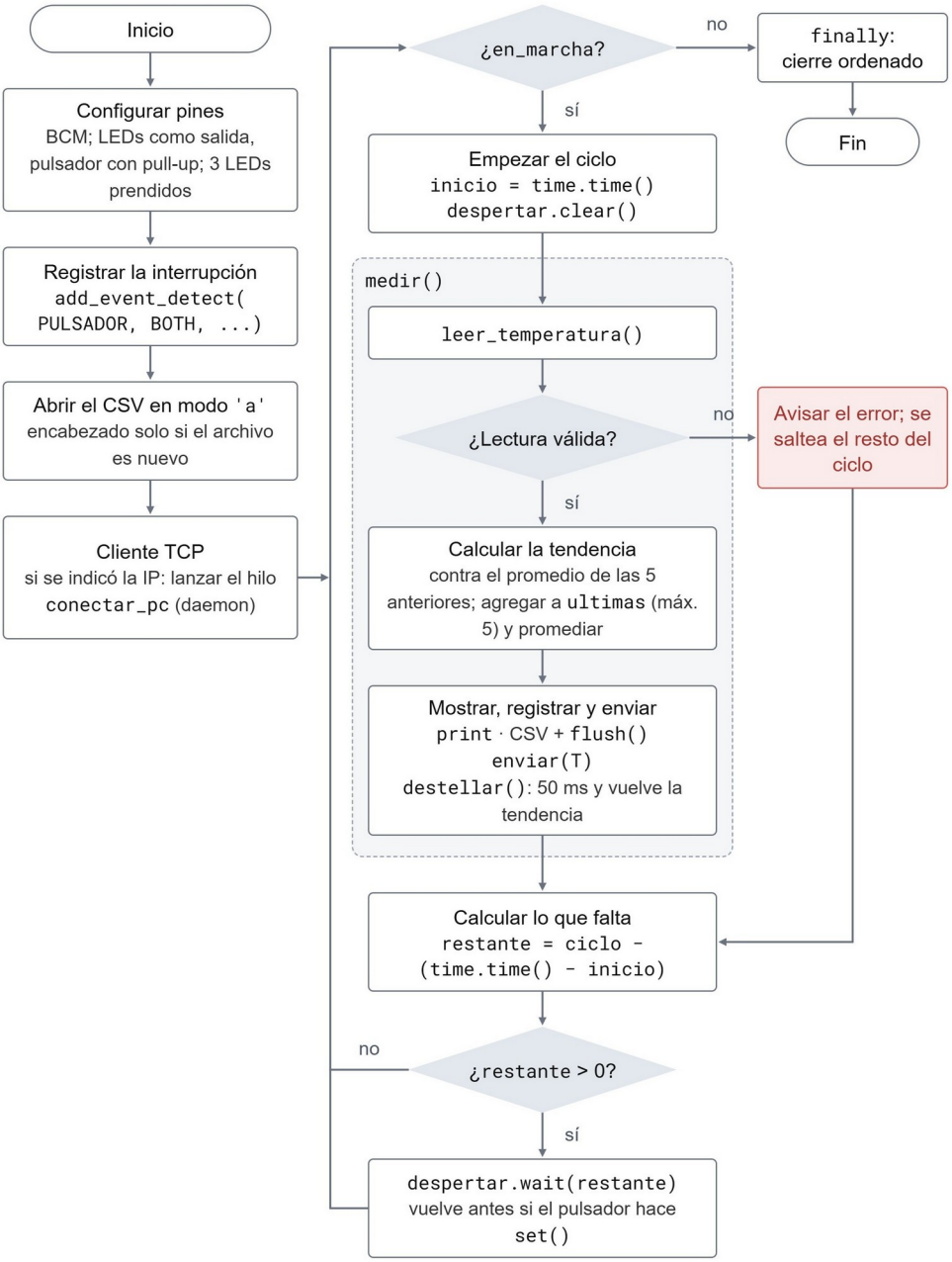


Figura 7. *Hilo principal. El recuadro punteado es* `medir()`*; si el sensor falla, el ciclo se saltea pero la espera se respeta.*

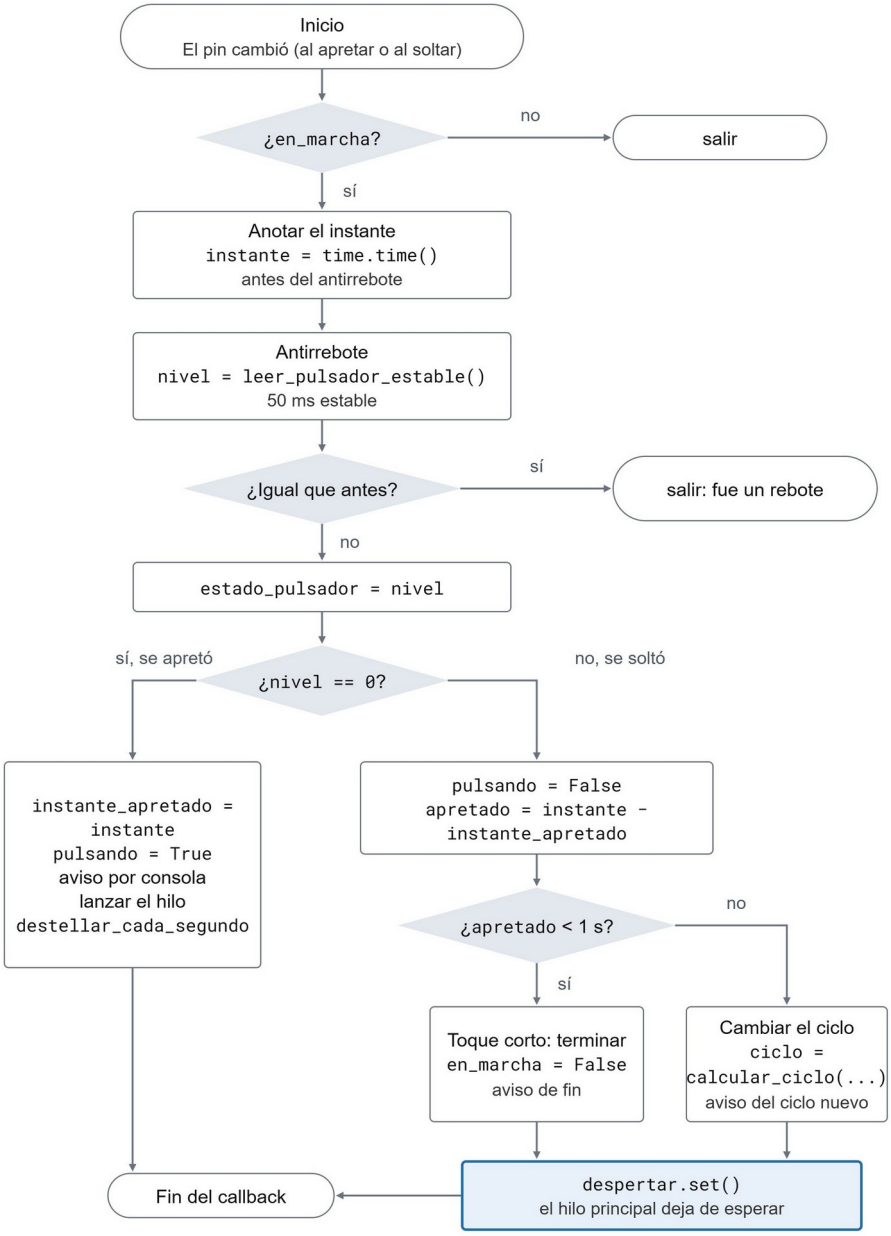


Figura 8. *Interrupción del pulsador (`al_cambiar_pulsador`). El ciclo se decide recién al soltar; la caja resaltada conecta este hilo con el principal.*

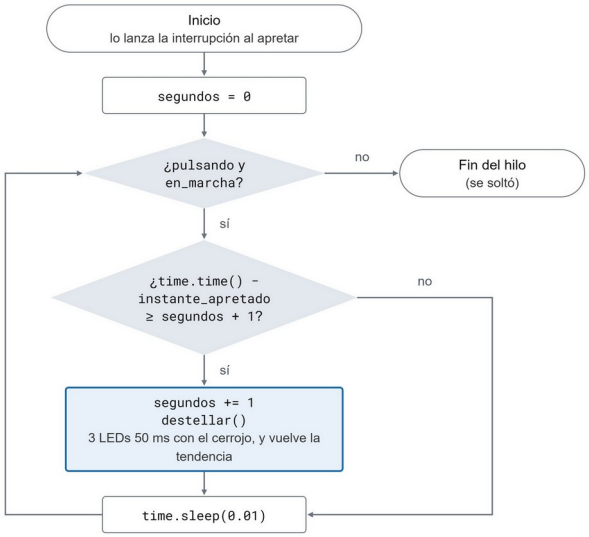


Figura 9. *Destellos por segundo: cuenta desde* `instante_apretado` *y no con* `sleep(1)`*, así los destellos no se atrasan.*

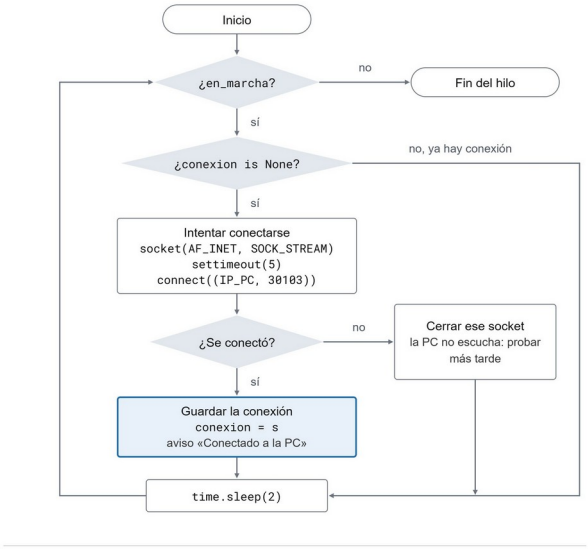


Figura 10. *Hilo de red (Parte C):* `connect()` *puede tardar hasta 5 s, por eso no está en el hilo principal.*

Código en Python

Librerías utilizadas

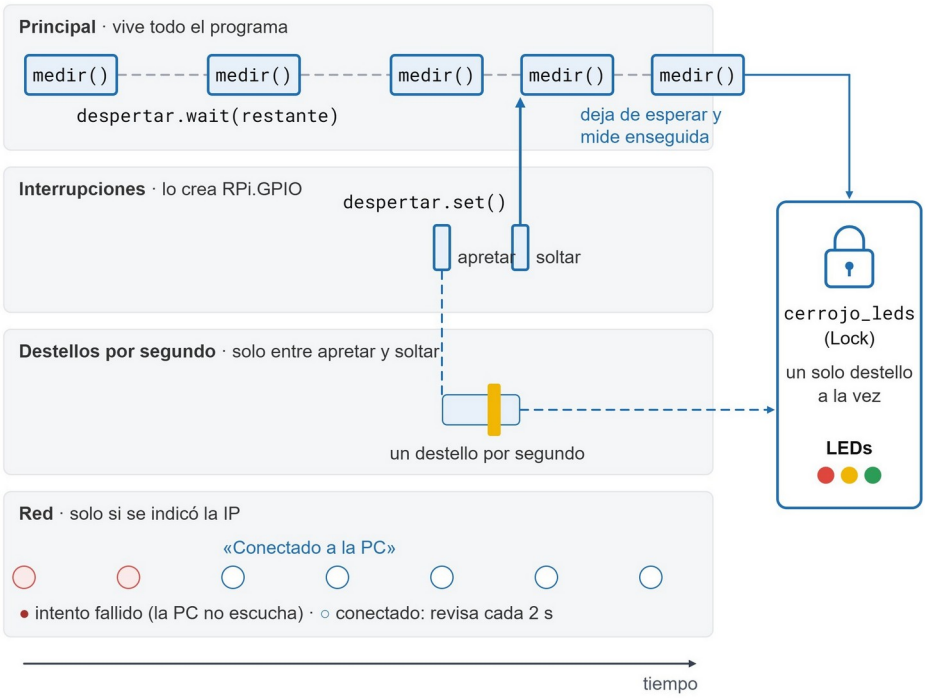
Todas vienen con Python o con Raspberry Pi OS. `socket` y `struct` son de la Parte C.

Librería	Qué usé	Para qué
RPi.GPIO	setmode, setup, output, input, add_event_detect, remove_event_detect, cleanup	LEDs, pulsador e interrupción; liberar los pines. La vista en clase.
threading	Thread(daemon=True), Lock(), Event()	Hilos de destellos y de red; proteger los LEDs; despertar al hilo principal.
time	time(), sleep()	Medir el tiempo apretado y la duración de la medición; los 50 ms.
datetime	datetime.now()	Fecha y hora de cada lectura.
os	os.path.exists()	Escribir el encabezado del CSV solo la primera vez.
glob (sensor.py)	glob('/sys/bus/w1/devices/28-*')	Encontrar la carpeta del sensor sin escribir su número de serie.

Hilos, variables compartidas y sincronización

El programa usa un hilo principal para medir, otro de RPi.GPIO para atender el pulsador, uno para los destellos mientras se mantiene apretado y otro para conectarse a la PC. Este último solo se inicia si se indicó una IP. Los hilos que creé usan daemon=True para que no mantengan abierto el programa cuando termina el principal.

Usé hilos porque algunas tareas tienen esperas. Por ejemplo, leer el sensor tarda unos 0,8 s y una conexión puede tardar hasta 5 s. Si todo se hiciera seguido en un mismo hilo, el pulsador podría responder tarde. Separar las tareas ayudó a resolver eso, aunque después tuve que coordinar el acceso a los LEDs y el cambio de ciclo.



Variables compartidas

Variable	La escribe	La lee
`ciclo`	interrupción	principal
`en_marcha`	interrupción y `finally`	todos
`tendencia`	principal	`destellar()`
`conexion`	red y principal	principal

Esquema sin escala exacta: ciclo de 3,5 s; el pulsador se aprieta 1,5 s y el ciclo pasa a 2,5 s (el mínimo).

Figura 11. *Cuatro hilos trabajan a la vez; un evento y un cerrojo los coordinan. Abajo, las variables que comparten.*

threading.Event permite cortar la espera entre lecturas. El principal espera con despertar.wait(restante) y la interrupción llama a despertar.set() al soltar el pulsador. Si en ese momento se estaba leyendo el sensor, primero termina esa lectura y después comienza la siguiente sin hacer la espera habitual.

El Lock se usa para que dos hilos no cambien los LEDs durante el mismo destello. destellar() toma el cerrojo mientras trabaja y lo libera al terminar. mostrar_tendencia() no vuelve a tomarlo, porque también se llama desde destellar() y eso bloquearía el hilo.

En una versión anterior, estacion3.py, usé una variable llamada cambio_de_ciclo. Durante las pruebas el comportamiento al soltar el pulsador no era siempre el mismo. En un caso quedaron dos lecturas separadas por 0,86 s. Cambié esa parte por un Event y dejé definido que, al soltar, se termina la lectura en curso y se vuelve a medir sin esperar.

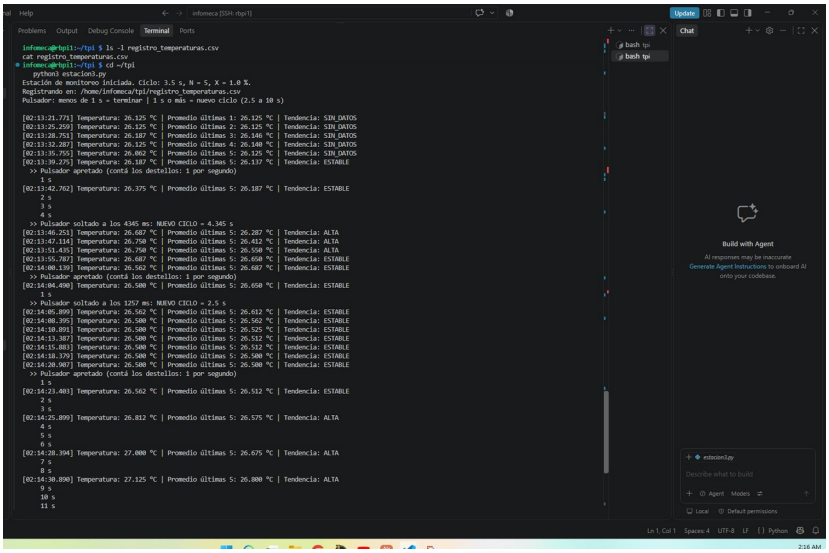


Figura 12. *estacion3.py: dos lecturas a 0,86 s (46,251 y 47,114).*

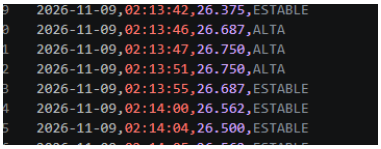


Figura 13. *El mismo caso en el CSV.*

Descripción de las subrutinas

Organicé el programa con funciones, sin definir clases. Cada una se encarga de una parte del funcionamiento.

- **leer_temperatura()** (sensor.py) → °C o None. Busca 28-*, abre w1_slave con with, verifica YES y convierte t=. Separarla en un módulo permite probarla sola (python3 sensor.py) y cambiar de sensor tocando un solo archivo.
- **poner_leds(r, a, v)**, **mostrar_tendencia()**: escriben los tres pines de una vez; la segunda prende solo el LED de la tendencia, o los tres si es SIN_DATOS.
- **destellar()**: con el cerrojo, invierte los LEDs 50 ms y vuelve a la tendencia.
- **calcular_tendencia(t)**: con menos de N lecturas previas, SIN_DATOS; si no, compara con promedio × (1 ± X/100). Se llama **antes** de agregar la lectura a la lista.
- **calcular_ciclo(s)**: menos de 2,5 → 2,5; 10 o más → 10; entre medio, el mismo tiempo redondeado a centésimas.
- **leer_pulsador_estable()**: antirrebote; lee cada 5 ms y devuelve el nivel cuando quedó igual 50 ms seguidos.
- **al_cambiar_pulsador(canal)**: el callback. Toma el instante **antes** del antirrebote para no sumarle 50 ms al tiempo medido.
- **destellar_cada_segundo()**: el hilo de destellos; revisa cada 10 ms si pasó un segundo más.
- **medir()**: un ciclo completo: leer, tendencia, promedio, consola, CSV con flush(), envío y destello.
- **Programa principal**: calcula restante = ciclo − (time.time() − inicio) para descontar los 0,8 s del sensor. Midiendo con milisegundos vi que antes el ciclo daba 3,52 s.

El cierre está en un bloque finally para liberar los recursos también cuando se interrumpe el programa. Tuve que revisar el orden porque en estacion4.py apareció el error 'unknown handle': una interrupción se ejecutó después de liberar los GPIO.

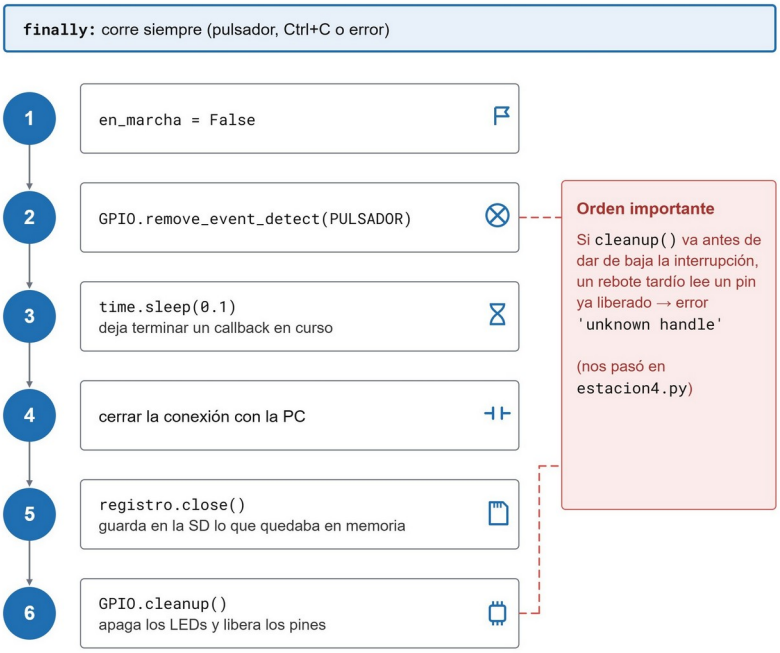


Figura 14. *Termine como termine (pulsador, Ctrl+C o error), finally libera todo en el mismo orden.*

Registro de pruebas previas a la presentación

La prueba previa a la presentación duró unos 5 minutos, de las 01:11 a las 01:16. Al iniciar quedaron los tres LEDs prendidos y las primeras 5 lecturas se registraron como SIN_DATOS. Al calentar el sensor con los dedos, a las 01:11:44 pasó a ALTA y se prendió el rojo (30,937 °C > 30,162 × 1,01 = 30,464 °C). Cuando se enfrió, a las 01:12:02 pasó a BAJA y se prendió el verde (31,562 < 31,975 × 0,99 = 31,655 °C). Si la temperatura deja de cambiar, vuelve a ESTABLE aunque siga siendo alta.

Tiempo apretado	Ciclo nuevo	Qué observé
1,47 s	2,5 s (mínimo)	Medición inmediata al soltar
9,15 s	9,15 s	Soltado durante una lectura: dos lecturas seguidas, separadas por 1 s (01:13:36 y 01:13:37), después cada 9 s
14,27 s	10,0 s (máximo)	Medición inmediata y después cada 10 s
0,12 s	—	Fin., cierre limpio

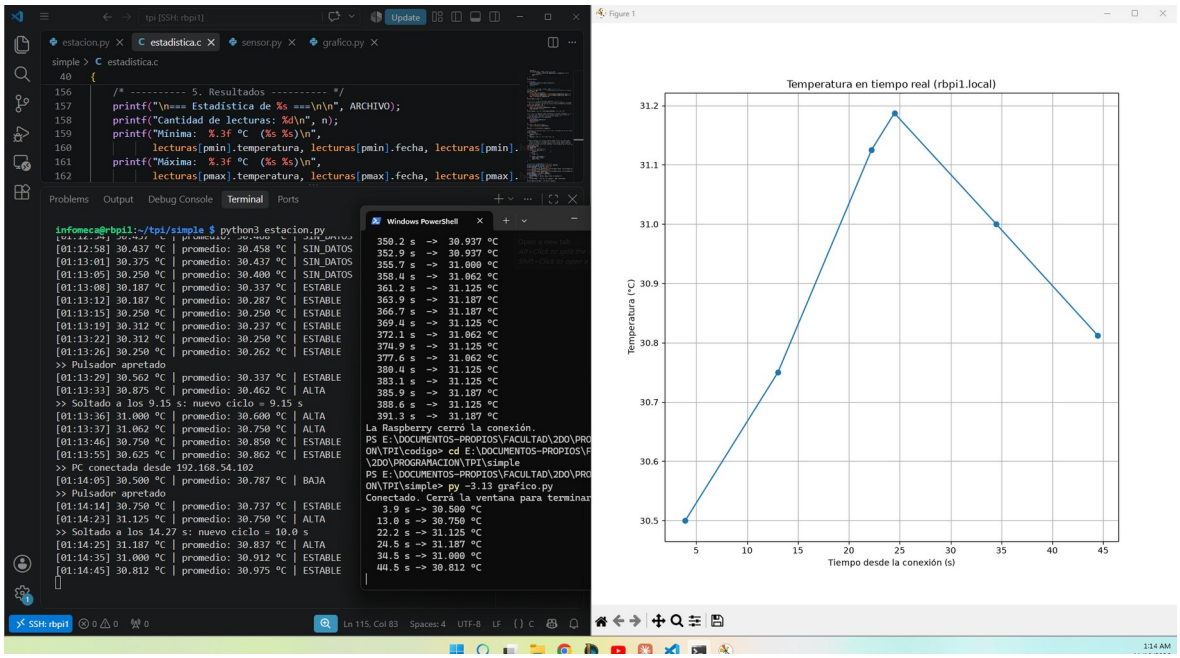


Figura 15. *Soltado a los 9,15 s (lecturas a las 01:13:36 y 01:13:37) y a los 14,27 s (ciclo de 10 s). A la derecha, el gráfico de la Parte C.*

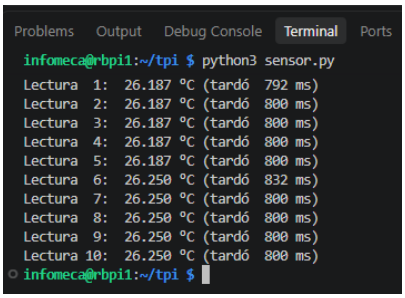


Figura 16. *python3 sensor.py: lecturas de unos 800 ms.*

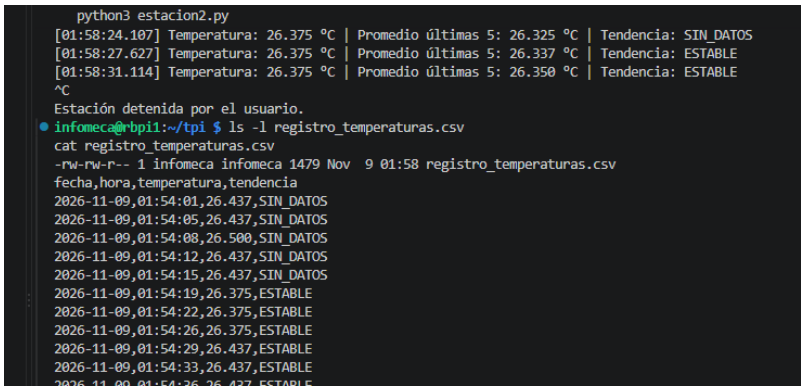


Figura 17. *ls -l y contenido de registro_temperaturas.csv (fecha,hora,temperatura,tendencia).*

Parte B

Para procesar los datos que entrega la estación de monitoreo se requiere hacer un programa que realice un análisis estadístico de las lecturas realizadas. El programa toma el archivo generado por el sistema y realiza las siguientes operaciones:

- Imprime la cantidad de lecturas realizadas por la estación.
- Imprime la menor y la mayor temperatura detectada, junto a la fecha y hora de la muestra.
- Calcula e imprime la media, mediana, moda y alguna medida de dispersión de los datos (ej. desviación estándar o varianza). Justifique qué técnica de dispersión se adapta mejor a los datos producidos.

Sobre la resolución: el ejercicio debe ser codificado en C; diseñe y utilice una estructura que modele los datos; para la gestión de los datos utilice memoria dinámica. Se realiza en la misma Raspberry anterior.

Introducción

Para esta parte hice estadistica.c, que lee las temperaturas guardadas y muestra los resultados por consola. Se ejecuta en la Raspberry y se compila con gcc -Wall -o estadistica estadistica.c -lm. La opción -lm permite usar sqrt(). En la compilación no aparecieron advertencias.

Requerimientos y restricciones

Entrada: registro_temperaturas.csv, con encabezado y una línea por lectura. **Salida:** cantidad de lecturas, mínima y máxima con fecha y hora, media, mediana, moda y desvío estándar.

- En C, sin librerías externas, con un struct y memoria dinámica: no se sabe de antemano cuántas lecturas hay.
- Toda la memoria se devuelve con free y el archivo se cierra con fclose.
- El programa controla si falta el archivo o si no hay lecturas. También saltea las líneas que no tienen el formato esperado.
- Se cuentan todas las lecturas válidas, incluidas las SIN_DATOS: les falta la tendencia, no el dato.
- Para el desvío usé la fórmula muestral, que divide por n − 1. Si hay una cantidad par de datos, la mediana es el promedio de los dos centrales. Si hay empate en la moda, se toma el menor valor.

Estructura de datos y memoria dinámica

```
typedef struct {
   char fecha[11];       /* "2026-11-09" = 10 caracteres + '\0' */
   char hora[9];        /* "01:50:24"   =  8 caracteres + '\0' */
   double temperatura;
   char tendencia[10];    /* "SIN_DATOS"  =  9 caracteres + '\0' */
} Lectura;
```

La fecha, la hora y la tendencia dejan un carácter extra para '\0', que marca el final del texto. El arreglo empieza con espacio para 16 lecturas y se amplía con realloc cuando se llena. Para las 439 lecturas se necesitaron 5 ampliaciones. Usé un puntero auxiliar para no perder el arreglo original si realloc falla. Para calcular mediana y moda se ordena una copia de las temperaturas. Al terminar se liberan los dos arreglos con free.

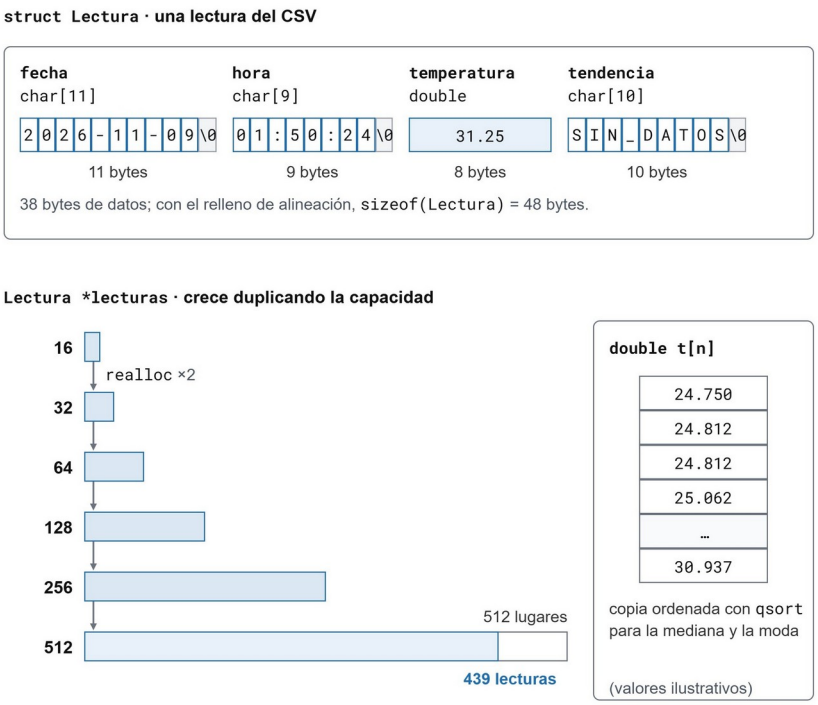


Figura 18. *Las lecturas viven en un arreglo de* `struct Lectura` *que crece con* `realloc`.

Algoritmo general de alto nivel

1. **Cargar el archivo:** abrirlo (si no existe, avisar y terminar), pedir memoria, saltear el encabezado y guardar cada línea válida, duplicando el arreglo cuando se llena.
2. **Mínima, máxima y media** en un recorrido, guardando las **posiciones** de la mínima y la máxima.
3. **Desvío estándar** en un segundo recorrido (necesita la media ya calculada).
4. **Mediana y moda:** ordenar la copia con `qsort`; la moda es la racha más larga de valores iguales.
5. **Mostrar los resultados** y liberar la memoria.

Diagrama de flujo

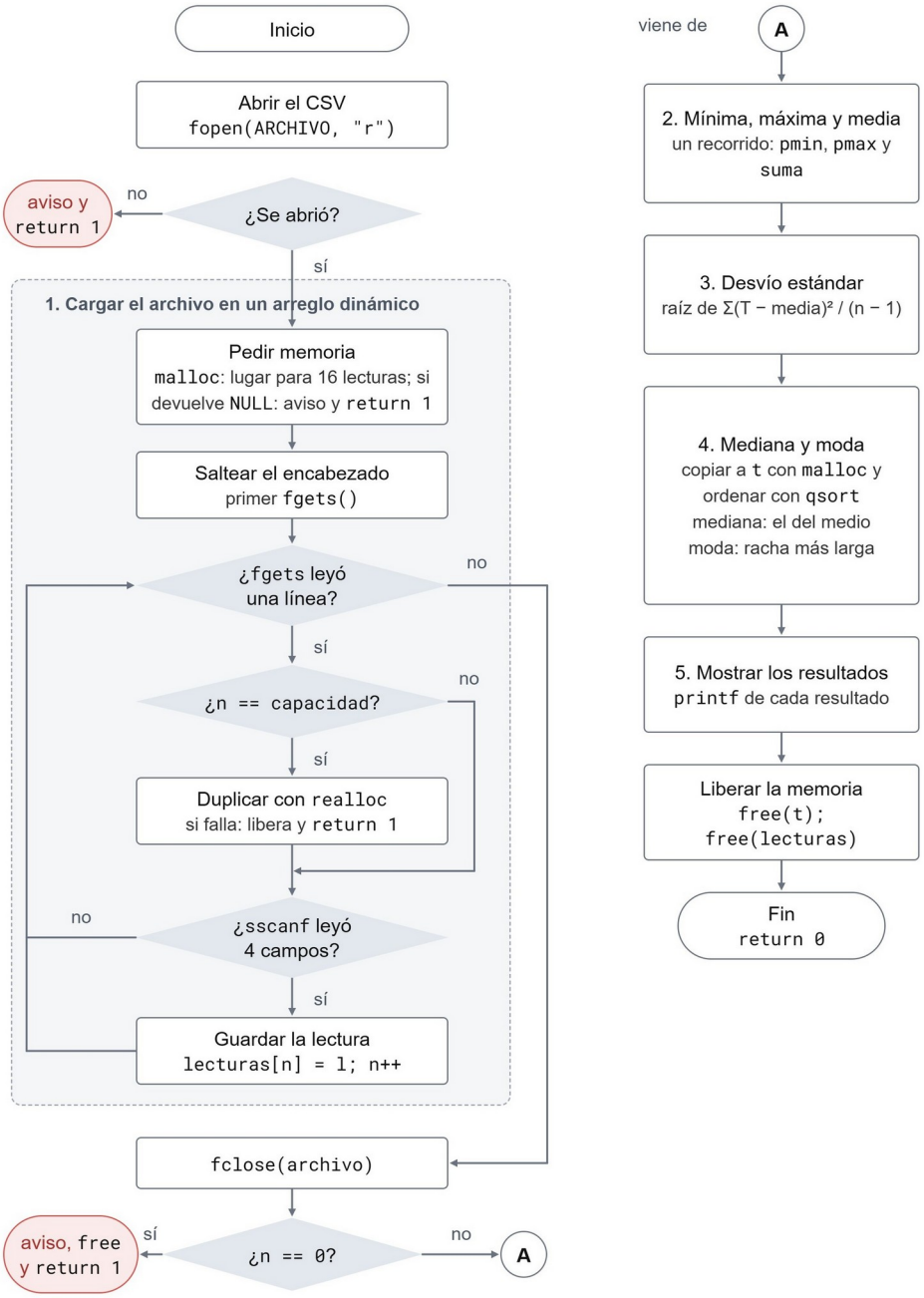


Figura 19. *estadistica.c. El recuadro punteado es el único bloque que toca el archivo; en rojo, los errores que el programa controla.*

Librerías utilizadas

Librería	Qué usé	Para qué
<stdio.h>	fopen, fgets, sscanf, printf, fclose	Leer el CSV línea por línea, separar los campos y mostrar los resultados.
<stdlib.h>	malloc, realloc, free, qsort	Memoria dinámica y ordenamiento.
<math.h>	sqrt	La raíz del desvío (por eso -lm).

fgets limita la cantidad de caracteres que se leen según el tamaño del búfer. En sscanf también se limita la longitud de los campos de texto para que entren en el struct. La línea se cuenta solo si se pudieron leer los 4 campos.

Descripción de las subrutinas

comparar(const void *a, const void *b) es la función que usa qsort para ordenar las temperaturas. Compara dos valores double y devuelve −1, 0 o 1 según su orden.

main() carga los datos, hace los cálculos y muestra los resultados. Para la moda se cuentan las repeticiones de cada temperatura en el arreglo ordenado. Si ocurre un error, se muestra un mensaje y se libera la memoria reservada. Cuando hay una sola lectura, el programa muestra desvío 0 para evitar dividir por cero; con un único dato no se puede calcular el desvío muestral.

Captura de pantalla demostrando el funcionamiento

Probé el programa con dos archivos. El primero tenía 439 lecturas de un día de pruebas: 119 de la madrugada, cerca de 26,5 °C, y 320 de la noche, cerca de 31 °C. El segundo tenía las 179 lecturas de las pruebas finales.

Resultado	CSV real	CSV de pruebas
Cantidad de lecturas	439	179
Mínima	26,062 °C (09/11 02:13:35)	29,937 °C (10/11 00:45:50)
Máxima	33,250 °C (09/11 23:41:03)	32,187 °C (10/11 01:11:51)
Media / mediana	30,193 / 31,062 °C	30,459 / 30,375 °C
Moda	31,687 °C (48 veces)	30,187 °C (26 veces)
Desvío estándar	**2,203 °C**	**0,410 °C**

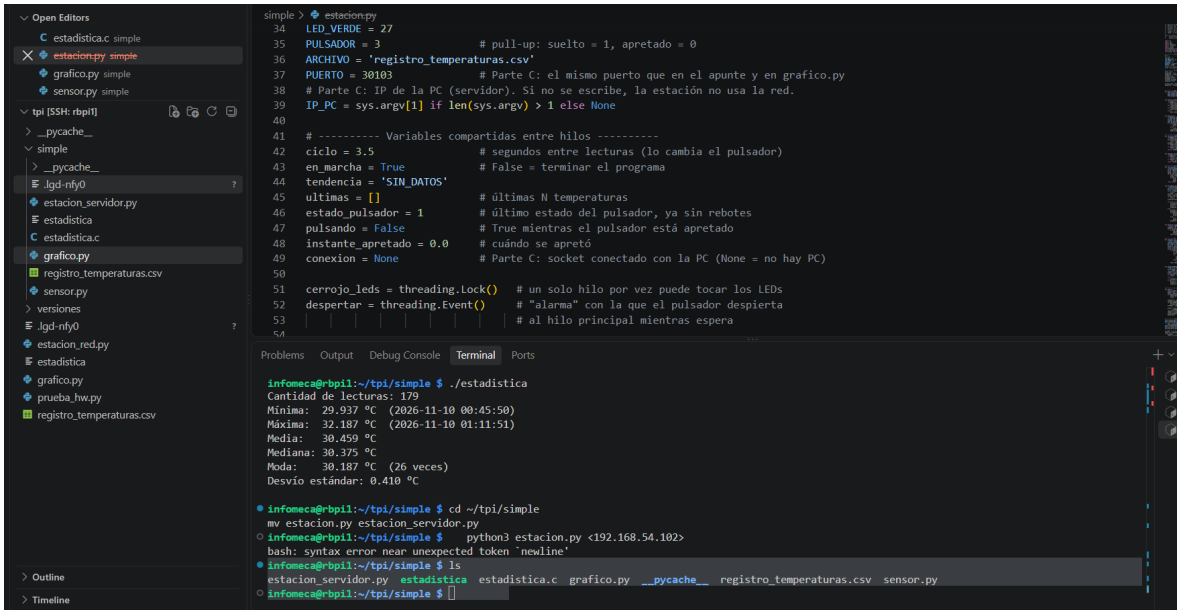


Figura 20. *./estadistica sobre el CSV de las pruebas: 179 lecturas, desvío 0,410 °C.*

Cómo se calcula el desvío estándar en el programa

$$s = \sqrt{\left(\Sigma \, (T_i - \bar{T})^2 / (n - 1) \right)}$$

El desvío es el bloque 3 de `main()`. Necesita la media, así que se calcula en un segundo recorrido del arreglo, después del bloque 2:

```
double suma_cuadrados = 0;
for (int i = 0; i < n; i++) {
   double d = lecturas[i].temperatura - media;
   suma_cuadrados += d * d;
}
double desvio = (n > 1) ? sqrt(suma_cuadrados / (n - 1)) : 0;
```

- Para cada lectura calcula la distancia a la media (`d`) y suma su cuadrado: así las distancias por arriba y por abajo no se cancelan.
- Se divide por n − 1 porque elegí tratar las lecturas como una muestra de la temperatura del ambiente.
- `sqrt()` viene de `<math.h>`; por eso se compila con `-lm`. La raíz devuelve el resultado a °C, la misma unidad que las lecturas.
- Si hay una sola lectura, `n − 1` sería 0: el operador `? :` evita la división y deja el desvío en 0.
- Al final se muestra con `printf("Desvío estándar: %.3f °C\n", desvio)`, con 3 decimales como las lecturas.

Parte C

Para analizar los datos que entrega la estación de monitoreo se requiere hacer un programa que realice una visualización gráfica de las lecturas realizadas. Si se utiliza Raspberry Pi, la comunicación con la PC se realizará mediante sockets TCP/IP. El programa consta de 2 partes:

- Un programa en Raspberry/PC principal en el cual se deberá configurar un socket TCP/IP y que, una vez establecido este, proceda a enviar las lecturas de los sensores por el mismo.
- Un programa en una computadora extra que se conectará al socket previamente establecido en la Raspberry Pi o PC principal.
- Proceder a la visualización de los datos en la computadora final, de manera gráfica (se sugiere la librería matplotlib), actualizándose a medida que se reciban los datos, es decir, en tiempo real.

Sobre la resolución: el ejercicio debe ser codificado en Python.

Introducción

En esta parte se muestra en la PC un gráfico con las temperaturas que manda la Raspberry. Usé dos programas conectados por TCP/IP, siguiendo el apunte de la cátedra “Uso de Socket TCP-IP”.

- **En la PC, `grafico.py` es el servidor:** abre el puerto 30103, espera la conexión y grafica cada dato con matplotlib. Se corre primero, con `py -3.13 grafico.py`.
- **En la Raspberry, `estacion.py` es el cliente:** la misma estación de la Parte A, con un hilo que se conecta a la PC. Se corre con `python3 estacion.py <IP de la PC>`.

Requerimientos y restricciones

- Los dos equipos tienen que estar en la misma red, y el firewall de Windows tiene que permitir conexiones entrantes a Python en el puerto 30103.
- La IP de la PC la asigna el router y puede cambiar: se pasa como argumento, no queda fija en el código. Sin IP, la estación funciona solo como Parte A.
- La estación no puede depender de la red: si la PC se desconecta, avisa, sigue midiendo y reintenta cada 2 s.
- La ventana del gráfico tiene que seguir respondiendo mientras espera datos, que llegan cada 2,5 a 10 s.
- Tomé a la PC como servidor y a la Raspberry como cliente, siguiendo el apunte. Por la red se envía la temperatura. El eje X muestra el tiempo transcurrido desde la conexión, medido en la PC.

Librerías adicionales utilizadas

Librería	Qué agrega	Para qué la usé
`socket`(Raspberry y PC)	Acceso a la red del sistema operativo: crear conexiones TCP/IP, esperar clientes y enviar o recibir bytes.	En la PC, abrir el servidor (`bind`, `listen`, `accept`) y recibir (`recv`). En la Raspberry, conectarse (`connect`, con `settimeout(5)`) y enviar cada temperatura (`sendall`).
`struct`(Raspberry y PC)	Convertir valores de Python en bytes con un formato fijo, y al revés.	`pack('!f', T)` convierte la temperatura en 4 bytes para enviarla; `unpack('!f', datos)` la vuelve a convertir en número en la PC.
`matplotlib.pyplot`(PC)	Gráficos en ventanas, con un modo interactivo que permite actualizarlos mientras el programa sigue corriendo. Es la única librería externa (`pip install matplotlib`).	Dibujar la curva en tiempo real: `ion()` activa el modo interactivo, `set_xdata`/`set_ydata` cambian los puntos, `relim()` y `autoscale_view()` ajustan los ejes y `pause(0.1)` redibuja la ventana.

Librería	Qué agrega	Para qué la usé
sys(Raspberry)	Datos del intérprete, entre ellos los argumentos escritos al correr el programa.	sys.argv[1]: la IP de la PC, para no dejarla fija en el código.
time(PC)	La hora del sistema en segundos.	time.time() al conectarse y en cada dato: el eje X son los segundos desde la conexión.

Configuración de red (sockets TCP/IP)

Para conectar los equipos usé TCP. En la PC se crea un socket con socket(AF_INET, SOCK_STREAM), se configura con bind(('0.0.0.0', 30103)) y se espera al cliente con listen(5) y accept(). La Raspberry se conecta con connect((IP_PC, 30103)) y un tiempo de espera de 5 s. TCP entrega los bytes en orden mientras la conexión funciona; si se corta, el programa tiene que volver a conectarse.

La temperatura se convierte a 4 bytes con struct.pack('!f', T). El formato indica un número real de 32 bits en orden de red. Para enviarlo usé sendall(). Del otro lado, los bytes se acumulan hasta completar 4 y recién ahí se convierten en una temperatura, porque una llamada a recv() puede recibir solo una parte del dato.

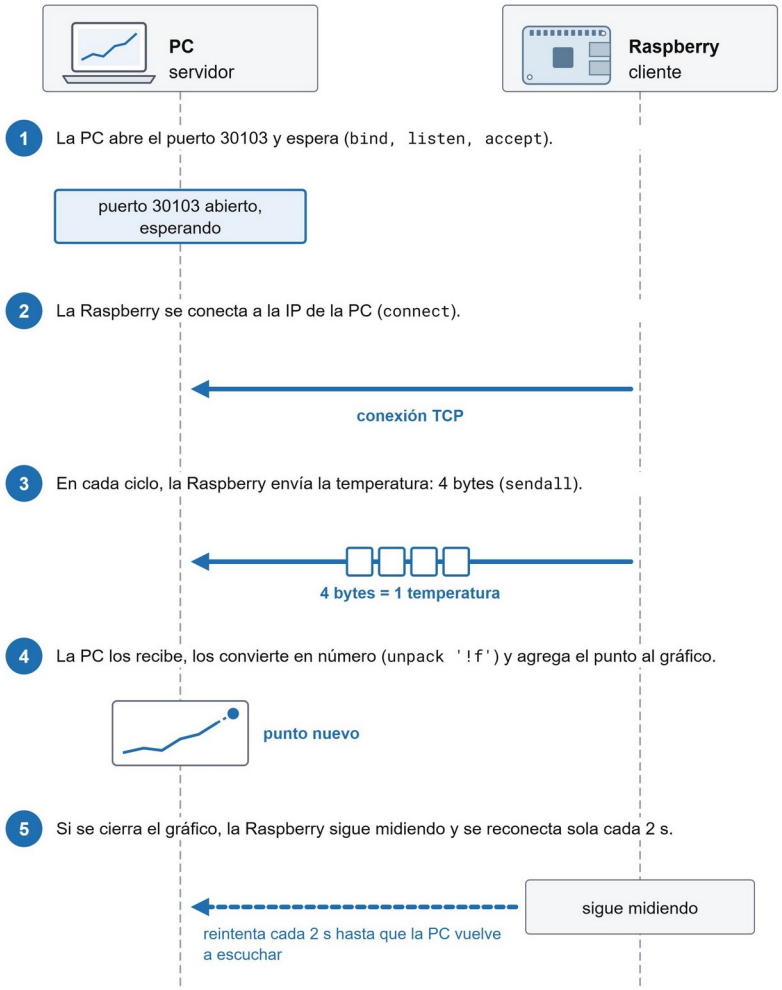


Figura 21. *La PC espera, la Raspberry se conecta y le manda cada medición.*

Algoritmo general de alto nivel

En la PC, el programa abre el socket, espera la conexión y prepara el gráfico. Después recibe los datos y agrega los puntos. El tiempo de espera de 0,1 s permite que la ventana siga respondiendo entre lecturas. Si la Raspberry cierra la conexión, se muestra un aviso y se sale del bucle.

En la Raspberry, conectar_pc() intenta conectarse cada 2 s cuando no hay conexión. En cada medición, enviar(T) manda la temperatura. Si el envío falla, se cierra el socket y conexion vuelve a None para que el hilo de red intente conectarse otra vez. El diagrama de este hilo está en la Parte A.

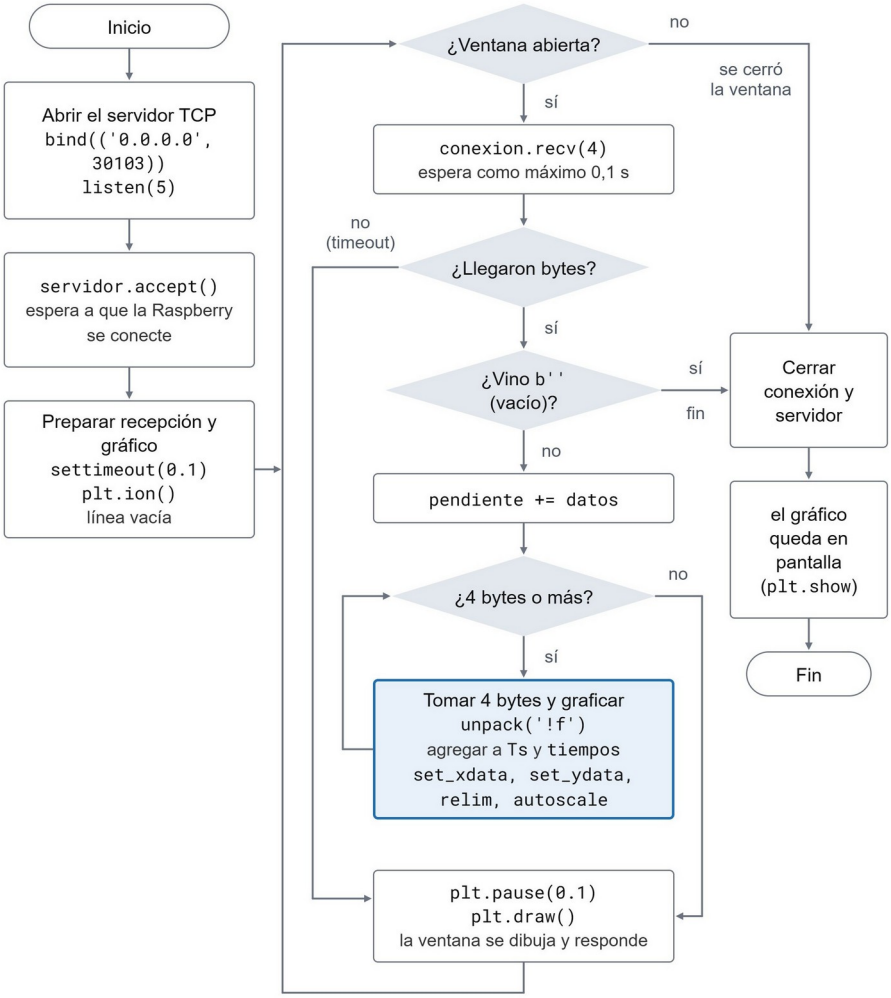


Figura 22. *grafico.py (servidor en la PC). El bucle de la derecha saca las temperaturas de a 4 bytes.*

Captura de pantalla demostrando el funcionamiento

A las 01:57, con ipconfig vi que la PC tenía la IP 192.168.54.102. Con grafico.py esperando, la Raspberry mostró “Conectado a la PC (192.168.54.102:30103)”. Al calentar el sensor, a las 01:58:16 la tendencia pasó a ALTA (31,250 °C > 30,462 × 1,01 = 30,767 °C) y el gráfico dibujó cada punto apenas llegó, igual que la consola.

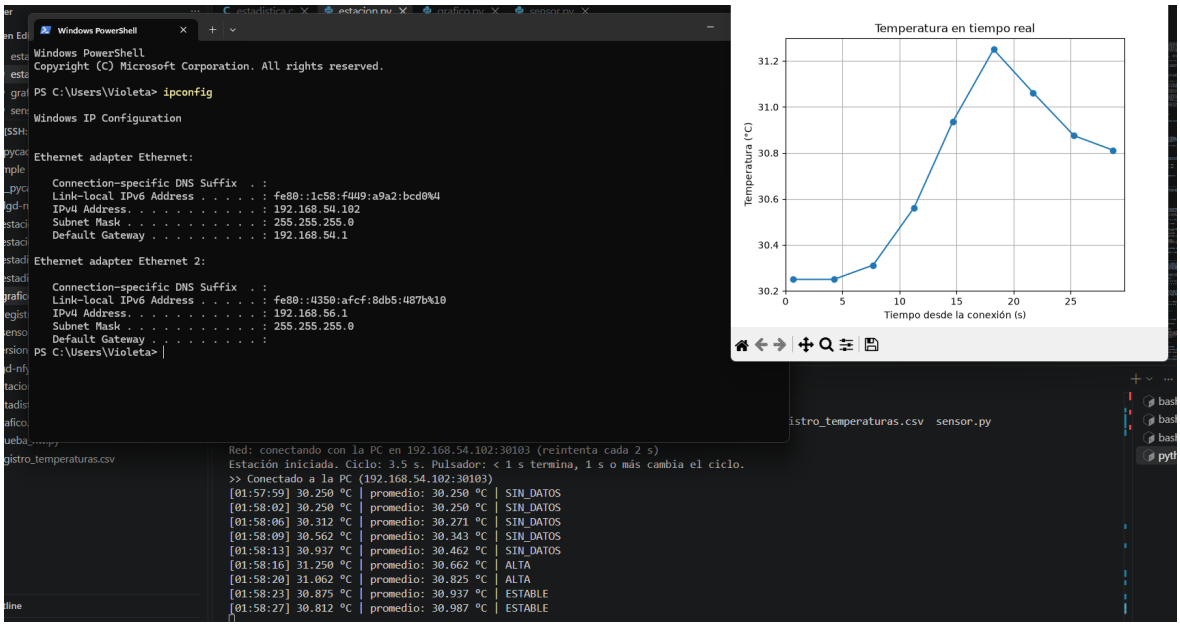


Figura 23. *ipconfig en la PC, la estación conectada a 192.168.54.102:30103 y el gráfico en tiempo real.*

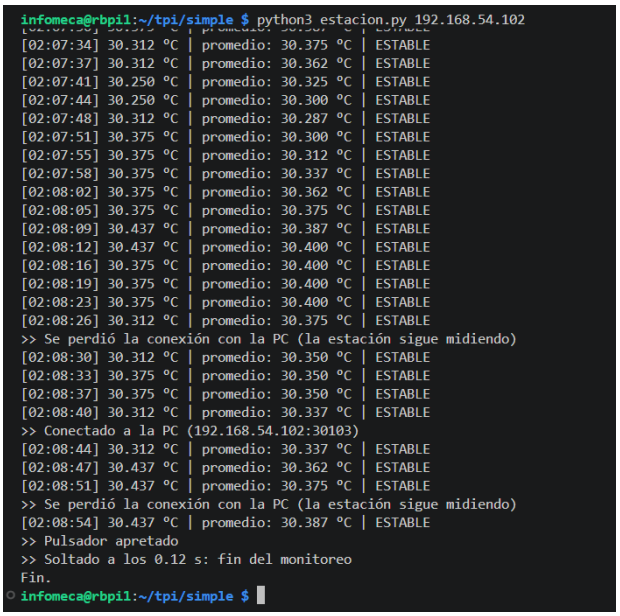


Figura 24. *Desconexión al cerrar el gráfico, reconexión sola y fin con el pulsador.*

Desconexión y reconexión (02:07 a 02:09). Al cerrar el gráfico, la estación mostró “Se perdió la conexión con la PC (la estación sigue midiendo)” y siguió registrando. Al volver a abrir grafico.py se reconectó sola, sin tocar la Raspberry. Un toque de 0,12 s terminó la estación con “Fin.”.

El aviso aparece un envío después del cierre: TCP informa el error recién cuando el otro lado responde que la conexión ya no existe.

Adicionales

Python de 32 bits en Windows

Al principio tuve problemas para instalar matplotlib porque en la PC estaba Python 3.14 de 32 bits. pip intentó compilar el paquete y apareció WinError 4551. Lo resolví instalando Python 3.13 de 64 bits con winget install --id Python.Python.3.13 -e --architecture x64. Después instalé matplotlib con py -3.13 -m pip install matplotlib.

Acceso remoto por SSH y mDNS

Para trabajar sin monitor usé SSH desde PowerShell con ssh infomeca@<IP> y la extensión Remote-SSH de VS Code. Como la IP puede cambiar, también usé el nombre rbpi1.local. Con ping -4 rbpi1.local pude consultar la IP de la Raspberry.

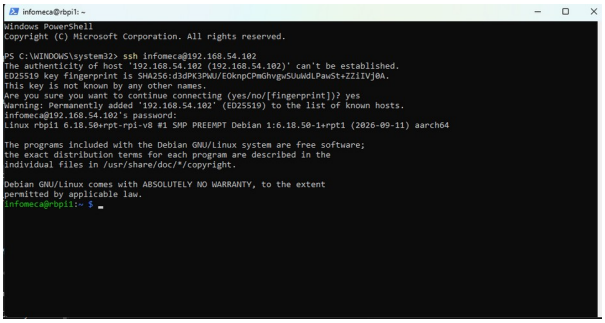


Figura 25. *Primera conexión por SSH desde PowerShell.*

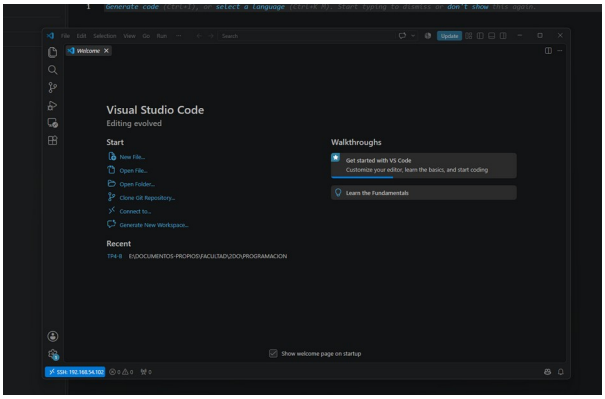


Figura 26. *VS Code conectado por Remote-SSH.*

Prueba final: notebook del coloquio y hotspot

Antes del coloquio probé el sistema con la notebook que iba a usar en la presentación, con Debian de 32 bits y matplotlib instalado desde apt. Conecté los equipos al hotspot del celular y agregué esa red a la Raspberry con nmcli. La estación se conectó a 10.147.150.167:30103. Al calentar el sensor, a las 03:33:06 pasó a ALTA (31,000 > 30,587 × 1,01 = 30,893 °C).

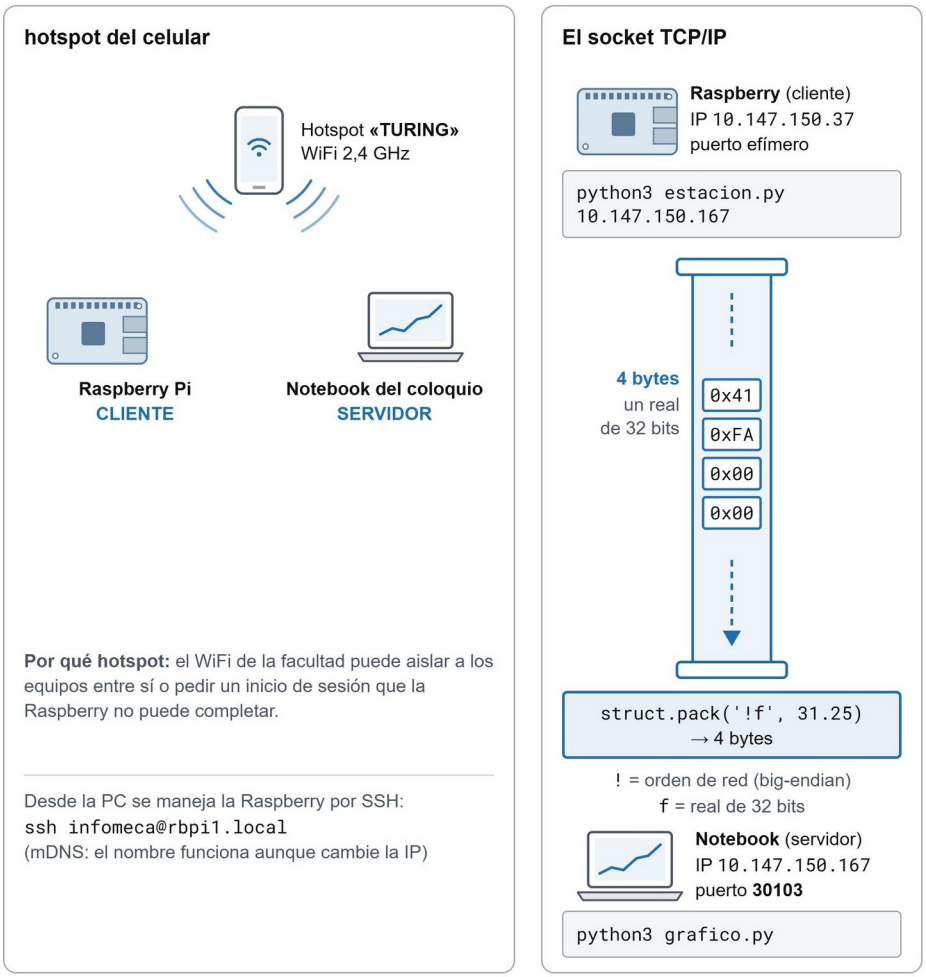


Figura 27. *La red del hotspot une los equipos; el socket TCP lleva los datos.*

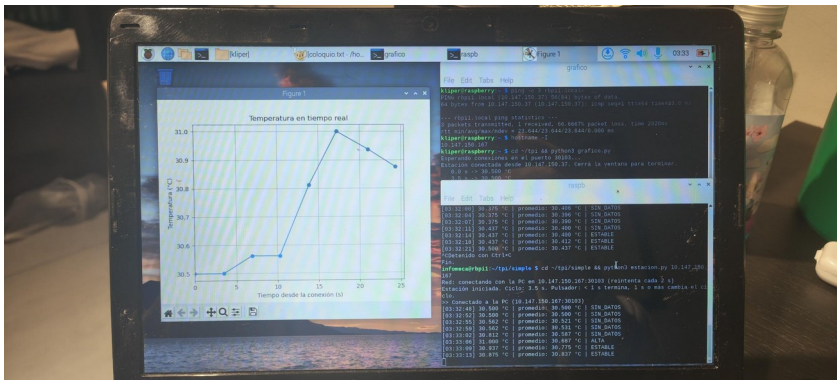


Figura 28. *La prueba en la notebook del coloquio: el gráfico en tiempo real y las terminales del servidor y de la Raspberry.*

Otra opción era usar gpiozero, que simplifica el manejo del pulsador y los LEDs. En este trabajo elegí RPi.GPIO porque es la librería que vi en clase.

Conclusiones

Cumplimiento de especificaciones

En las pruebas realizadas pude comprobar las funciones principales que pide la consigna. La tabla resume dónde se resolvió cada una.

Requisito	Cómo lo cumplí y evidencia
A – Ciclo de 3,5 s, lectura y promedio de 5	Bucle que espera solo lo que falta; `medir()` y la lista `ultimas`.
A – Ciclo con el pulsador (2,5 a 10 s) y fin	Interrupción + `calcular_ciclo()` + `Event`: 1,47 → 2,5 s; 9,15 → 9,15 s; 14,27 → 10 s; 0,12 s → Fin.
A – Semáforo y 3 LEDs sin datos	ALTA 01:11:44, BAJA 01:12:02, ESTABLE; 5 × `SIN_DATOS` al arrancar.
A – Destellos de 50 ms	`destellar()` con cerrojo y el hilo de destellos.
A – Registro en archivo	CSV en modo `'a'` con `flush()`: 439 lecturas reales.
B – Estadísticas y dispersión justificada	439 lecturas; mín. 26,062 °C y máx. 33,250 °C con fecha; desvío muestral.
B – Estructura y memoria dinámica	`struct Lectura`, `malloc/realloc/free`.
C – Socket TCP/IP y gráfico en tiempo real	“Conectado a la PC”; curva de 30,25 a 31,25 °C al calentar el sensor.

Consideraciones finales sobre el funcionamiento

El sistema permitió medir la temperatura, guardar los datos y verlos en un gráfico desde otra computadora. La parte que más tuve que revisar fue el pulsador, sobre todo cuando se soltaba durante una lectura. También aparecieron errores al cerrar el programa, que pude corregir cambiando el orden de cierre. Las pruebas con el circuito ayudaron a encontrar problemas que no había visto al revisar el código.

Quedaron algunas cosas para mejorar. El CSV acumula datos de distintas ejecuciones, por lo que las estadísticas pueden mezclar sesiones. Además, el archivo se crea en la carpeta desde donde se inicia el programa y la IP de la PC se escribe a mano. Si falla una lectura del sensor, ese ciclo se pierde porque no se vuelve a intentar.